



crée un site internet avec un presentation type : <https://www.surfiteasy.pt/>

1 -le site auras 4 fonction principale

A) l'admin et les moderateur :

- valide les comptes utilisateur, donne des credits aux utilisateur,
- crée un contenu planning a la semaine pour y placer cours de surf, de skate. velo tour et tourisme divers a cree
- ces evenement du planning seront sur une page internet type : 1.png et a partir de cet page les utilisateurs selection leurs cours et depense leurs credits pour participer a ce cours.

- B) un planning vers lequel les utilisateurs soit unique sois fesant partie de l'acad
- C) les moderateur qui seront aussi les professeur qui assure les cours peuvent voir
- D) une 4eme partie que vera le jours bientot pour assuré un fonction surf camp

le tout dois tourner de un conteneur Docker pour etre envoyer dans un VPS en ligne.

2 -cree les fichiers

ARCHITECTURE

DECISIONS

README_PROJECT

ROADMAP

TODO_NEXT

afin de cree un github pour developper la suite.

J'ai créé une première base de projet pour ton site, avec une page web de démonstration inspirée d'une surf school moderne, un conteneur Docker Nginx, et les 5 fichiers de cadrage demandés pour démarrer proprement un dépôt GitHub. Le design et la structure suivent les bonnes pratiques d'une web app avec dashboard, rôles, planning et réservation, notamment une navigation latérale, une hiérarchie compacte pour l'interface, et une architecture prête à évoluer vers un backend Node.js/PostgreSQL conteneurisé.^[1]

Contenu créé

La page `surf-academy-platform.html` montre une direction produit claire avec les 4 fonctions métier que tu as décrites : gestion admin/modérateur, planning hebdomadaire, réservation par crédits, et futur module surf camp. Elle inclut aussi une grille de planning type semaine, une vue dashboard, un thème clair/sombre, et une présentation pensée comme base de démonstration produit.^[1]

Les fichiers ARCHITECTURE, DECISIONS, README_PROJECT, ROADMAP et TODO_NEXT structurent le projet pour GitHub et posent les choix de base : Node.js/Express pour l'API, PostgreSQL pour les données transactionnelles, séparation wallet/réservations, rôles simples au départ, et approche Docker-first pour le VPS. ^[1]

Structure proposée

L'architecture documentaire pose un MVP en 3 briques principales : comptes et rôles, wallet de crédits, et sessions d'activités réservables, avec une extension surf camp prévue en phase 2. Ce découpage correspond bien aux recommandations pour une web app orientée dashboard, où la navigation, les rôles, les vues de gestion et les flux métier doivent être clairs dès le départ. ^[1]

La base Docker fournie sert déjà le prototype statique via Nginx, ce qui te permet d'avoir un point de départ simple pour le déploiement sur VPS avant d'ajouter api/, db/ et éventuellement un reverse proxy plus complet. Les décisions de projet formalisent aussi le fait que le paiement en ligne vient après validation du cœur métier, ce qui réduit le risque de complexité prématurée. ^[1]

Fichiers livrés

- surf-academy-platform.html : prototype visuel du site/app. ^[1]
- docker-compose.yml et Dockerfile : base de conteneurisation pour lancement local et VPS. ^[1]
- ARCHITECTURE.md : vision technique et fonctionnelle. ^[1]
- DECISIONS.md : décisions structurantes du projet. ^[1]
- README_PROJECT.md : point d'entrée du repo. ^[1]
- ROADMAP.md : plan de livraison par phases. ^[1]
- TODO_NEXT.md : prochaines tâches concrètes. ^[1]

Étape logique suivante

La prochaine étape utile est de transformer ce prototype en vrai mono-repo avec public/, api/, db/ et infra/, puis de brancher un backend Express avec PostgreSQL autour des entités users, credit_transactions, activity_sessions, bookings et attendance, qui sont précisément les entités adaptées à ton besoin de réservation par crédits et gestion professeur/admin. ^[1]

Je peux ensuite te générer la **suite complète du repo**, avec arborescence réelle, backend Express, schéma PostgreSQL, routes API, .env.example, Nginx VPS et scripts Docker Compose prêts au dev.

✱